1. What is OOP?

OOP (Object-Oriented Programming) is a programming approach where we structure code using objects and classes.

It helps in organizing code in a real-world way. For example, in a banking system, we can create a User class with properties like name and balance, and methods like deposit and withdraw.

---

2. What are the 4 pillars of OOP?

The four pillars of OOP are:

Encapsulation, Abstraction, Inheritance, and Polymorphism.

These concepts help in making code reusable, secure, and easy to maintain.

---

3. What is a Class?

A class is a blueprint or template used to create objects.

For example, a Car class defines properties like color and speed, and behavior like start or stop.

---

4. What is an Object?

An object is an instance of a class.

For example, if Car is a class, then a specific car like “Red BMW” is an object.

---

5. Difference between Class and Object

A class is a blueprint, while an object is a real instance created from that blueprint.

Class is a logical entity, whereas object is a physical entity that occupies memory.

🔒 Encapsulation & Abstraction

---

6. What is Encapsulation?

Encapsulation means binding data and methods together in a class and restricting direct access using access modifiers.

For example, we make variables private and use getter/setter methods to control access, which improves security.

---

7. What are Access Modifiers?

Access modifiers define the accessibility of variables and methods.

public → accessible everywhere

private → accessible only inside the class

protected → accessible within class and derived classes

---

8. What is Abstraction?

Abstraction means hiding complex implementation details and showing only the necessary functionality.

For example, when we use a function, we don’t know how it works internally, we just use it.

---

9. Encapsulation vs Abstraction

Encapsulation focuses on protecting data using access control, while abstraction focuses on hiding complexity and showing only essential features.

---

10. Abstract class

Abstract classes are used to provide a base class from which other classes can be derived.

They cannot be instantiated and are meant to be inherited.

Abstract classes are typically used to define an interface for derived classes.

11. Static Keyword

Static Variables

Variables declared as static in a function are created & initialised once for the lifetime of the program. //in Function

Static variables in a class are created & initialised once. They are shared by all the objects of the class. //in Class

Static Objects

🔁 Inheritance

---

10. What is Inheritance?

Inheritance is a mechanism where one class acquires the properties and behavior of another class.

It helps in code reusability and reduces duplication.

---

11. Types of Inheritance in C++

Single, Multiple, Multilevel, Hierarchical, and Hybrid inheritance.

---

12. Use of Inheritance

It helps in code reuse, reduces redundancy, and improves maintainability.

🔄 Polymorphism

---

13. What is Polymorphism?

Polymorphism is the ability of objects to take on different forms or behave in different ways depending on the context in which they are used.

---

14. Types of Polymorphism

Compile-time polymorphism → Function overloading, Operator overloading → statically, parmeter are different

Runtime polymorphism → Function overriding using virtual functions

---

15. Function Overloading

Function overloading means having multiple functions with the same name but different parameters.

---

16. Function Overriding(inheritence)(dynimcally)

Parent & Child both contain the same function with different implementation.

The parent class function is said to be overridden.

---

17. Virtual Function

A virtual function is used to achieve runtime polymorphism. It ensures that the correct function is called based on the object type.

Virtual functions are Dynamic in nature.

Defined by the keyword "virtual" inside a base class and are always declared with a base class and overridden in a child class.

⚙️ Advanced

---

18. Constructor

A constructor is a special function that is automatically called when an object is created.

It is used to initialize object properties and has the same name as the class with no return type.

Copy constructor → used to copy of one object into another`Teacher t2(t1)`

---

19. Destructor

A destructor is a special function that is called when an object is destroyed.

It is used to free resources and clean up memory. Syntax ~Teacher() {delete ptr}

---

20. this Pointer

The this pointer is a special pointer in C++ that refers to the current object.

It is used to access the current object's properties and methods.

Shallow & Deep Copy

A shallow copy of an object copies all of the member values from one object to another. Shared same memory affected eachother

A deep copy, on the other hand, not only copies the member values but also makes copies of any dynamically allocated memory that the members point to.

Does not affect each other

A car is a real-world example of OOP.

Its properties are color, speed, and model, and its methods are start, stop, and accelerate.

This shows how OOP represents real-world entities using objects and classes.

🔄 1. Overloading vs Overriding

Function Overloading and Function Overriding are both types of polymorphism.

Function Overloading:
It occurs when multiple functions have the same name but different parameters.
It is resolved at compile time.

Example:
int add(int a, int b);
int add(int a, int b, int c);

Function Overriding:
It occurs when a derived class redefines a function of the base class.
It is resolved at runtime using virtual functions.

Example:
Base class function is overridden in child class using virtual keyword.

👉 **One-line difference:**

Overloading = same function name, different parameters (compile-time)
Overriding = same function, different implementation in child class (runtime)

⚡ 2. Compile-time vs Runtime Polymorphism

Compile-time polymorphism is achieved during compilation.
It includes function overloading and operator overloading.

Runtime polymorphism is achieved during execution.
It includes function overriding using virtual functions.

👉 **Difference:**

Compile-time polymorphism is faster because it is resolved early,
while runtime polymorphism is flexible but slightly slower due to dynamic binding.

🧱 3. Abstract Class vs Interface

Abstract class:
It can have both abstract methods and normal methods.
It can have constructors and member variables.

Interface (in general concept):
It contains only abstract methods (no implementation).
Used to achieve full abstraction.

👉 **Simple difference:**

Abstract class = partial abstraction
Interface = full abstraction

👉 ⚠️ Interview Tip (IMPORTANT):

- In **C++**, there is no keyword “interface”
- We use **pure virtual class** as interface

🆕 4. new vs malloc (C++)

new:
Used in C++
Allocates memory and also calls constructor
Returns typed pointer

malloc:
Used in C
Only allocates memory (no constructor call)
Returns void pointer

👉 **Difference (short):**

new = memory allocation + constructor call
malloc = only memory allocation

🗑️ 5. delete vs delete[]

delete:
Used to free memory allocated for a single object.

delete[]:
Used to free memory allocated for an array of objects.

👉 **Example:**

```
int* p = new int;
delete p;

int* arr = new int[5];
delete[] arr;
```

🔥 FINAL QUICK REVISION (VERY IMPORTANT)

👉 Memorize these one-liners:

Overloading = compile-time, different parameters
Overriding = runtime, same function in child

Compile-time = fast, early binding
Runtime = flexible, late binding

Abstract class = partial abstraction
Interface = full abstraction

new = constructor + memory
malloc = only memory

delete = single object
delete[] = array